

Relatório Técnico - Mini E-commerce Distribuído

1. Como a comunicação entre os microsserviços foi implementada? A comunicação inter-serviços foi implementada de forma síncrona utilizando o padrão arquitetural REST sobre o protocolo HTTP. O sistema possui um **API Gateway** atuando como proxy reverso, que recebe as chamadas externas e roteia internamente para as portas específicas de cada microsserviço. Além disso, existe comunicação direta (coreografia) entre serviços de domínio: quando uma requisição de compra é feita, o Serviço de Pedidos realiza uma chamada HTTP GET interna (via biblioteca Axios) para o Serviço de Produtos, validando a existência do item e buscando seu preço atualizado antes de consolidar a ordem de compra.

2. Qual estratégia de consistência foi adotada na replicação? Forte ou eventual? Por quê? Foi adotada a estratégia de **Consistência Forte** (Strong Consistency). No Serviço de Produtos, que possui duas réplicas de armazenamento, toda operação de escrita (POST para criar um produto) só retorna a resposta de sucesso ao cliente (*201 Created*) após gravar os dados de forma síncrona e simultânea em ambas as réplicas (`products_replica1.json` e `products_replica2.json`). Essa decisão foi tomada porque, em um contexto de e-commerce, é crítico evitar anomalias no catálogo, como vender um produto fantasma ou com preço desatualizado. Priorizamos a exatidão e a integridade absoluta dos dados, mesmo que isso custe uma leve penalidade na latência de escrita. Para compensar, as leituras utilizam balanceamento de carga (*Round-Robin* simples), dividindo o fluxo de requisições entre as duas réplicas que estão garantidamente sincronizadas.

3. O que acontece com o sistema se o Serviço de Pedidos cair? O restante continua funcionando? A arquitetura garante alta tolerância a falhas parciais e o sistema continuará operando suas funções não afetadas. O API Gateway implementa um padrão ativo de verificação (*Heartbeat*), enviando requisições `GET /health` a todos os nós a cada 5 segundos. Se o Serviço de Pedidos cair e falhar em responder a duas tentativas consecutivas (timeout de 2s), o Gateway marcará o serviço como 'inativo' (*down*). A partir desse momento, qualquer tentativa de acesso à rota de pedidos será barrada precocemente no Gateway com o erro protetivo `503 Service Unavailable`, evitando sobrecarga ou que o cliente fique aguardando eternamente. Durante essa interrupção, o Serviço de Usuários (cadastro/login) e o Serviço de Produtos (pesquisa/criação) continuarão funcionando perfeitamente de forma independente.

4. Como o JWT garante que um usuário comum não consiga criar produtos? A segurança de autorização é baseada na verificação do *payload* do token JWT, que é assinado criptograficamente no momento do login pelo Serviço de Usuários usando uma chave secreta simétrica. O token carrega consigo os dados do usuário, incluindo a propriedade `role` (que pode ser 'user' ou 'admin'). Quando uma requisição para criar produtos é enviada através do Gateway, o token é repassado no cabeçalho

Authorization. O Serviço de Produtos intercepta essa chamada através de um *middleware*, decodifica o token validando sua assinatura e verifica o valor da *role*. Sendo um usuário comum (*role: 'user'*), o *middleware* aborta a operação e retorna o código HTTP **403 Forbidden**, impossibilitando a escalada de privilégios e mantendo a rota restrita aos administradores.

5. Quais limitações a sua implementação possui em relação a um sistema real de produção? Embora atenda aos princípios de sistemas distribuídos de forma didática, a implementação atual possui limitações críticas para um ambiente de produção em larga escala:

- **Persistência de Dados (Concorrência):** O uso de arquivos *.json* nativos não possui controle robusto de concorrência ou travas de I/O (*locks*). Múltiplas requisições simultâneas corromperiam os arquivos. Seria obrigatório o uso de bancos de dados preparados para transações (PostgreSQL, MongoDB, etc.).
- **Comunicação Puramente Síncrona:** A comunicação HTTP direta entre Pedidos e Produtos acopla os serviços no tempo. Se o Serviço de Produtos ficar lento, o de Pedidos também ficará. Em produção, adotaríamos filas e comunicação assíncrona (ex: RabbitMQ ou Apache Kafka) com o padrão *Event-Driven*.
- **Service Discovery Estático:** Os endereços dos serviços estão fixados no código (ex: *localhost:5001*). Em nuvem/clusters (Docker Swarm / Kubernetes), os IPs mudam dinamicamente, exigindo ferramentas de *Service Discovery* (como Consul ou Eureka) para roteamento dinâmico.